read at rate of 150 wrd/min

SPEAKING COPY

Paper to be printed in the Proceedings of the Eastern Joint Computer Conference, December 1-3, 1959

The Virtual Memory in the STRETCH Computer

John Cocke
and
Harwood G. Kolsky

International Business Machines Corporation
Poughkeepsie, New York

START of TALK

I. INTRODUCTION

Early in the planning of the STRETCH computer it was seen that by using the latest solid state components in sophisticated circuits that it would be possible to increase the speed of floating point arithmetic by almost two orders of magnitude over that in existing computers. However, there seemed to be no possibility of developing on the same time-scale economically feasible large memories with more than a factor of ten or perhaps twenty increase in speed. As a result, the proposed system appeared to be in danger of being seriously memory-access limited.

Moreover, as the speed of the floating point operations increases, a larger and larger percentage of the computer's time is spent on "parasitic operations", i. e., operations whose only function is program control and data selection. It was obvious that a radically new machine organization

12 min

was necessary in order to capitalize upon the possibilities opened up by the high arithmetic speeds in the presence of relatively slow memories.

At this time, a number of persons were considering the possibility of a "look-ahead" device in which an independent indexing arithmetic unit would prepare the effective addresses of instructions and initiate memory references to a multiplicity of memory boxes, so that several could be running simultaneously. ~~The data thus fetched would be held in high speed buffer registers until needed by the arithmetic unit.~~ This type of organization presented serious logical problems until the concept of the "virtual memory" was developed. ~~This device would serve two desirable purposes: (1) some of the parasitic operations would be done in parallel and thus not delay the principal calculations, and (2) several memory boxes could be running simultaneously, giving the effect of higher memory speed.~~

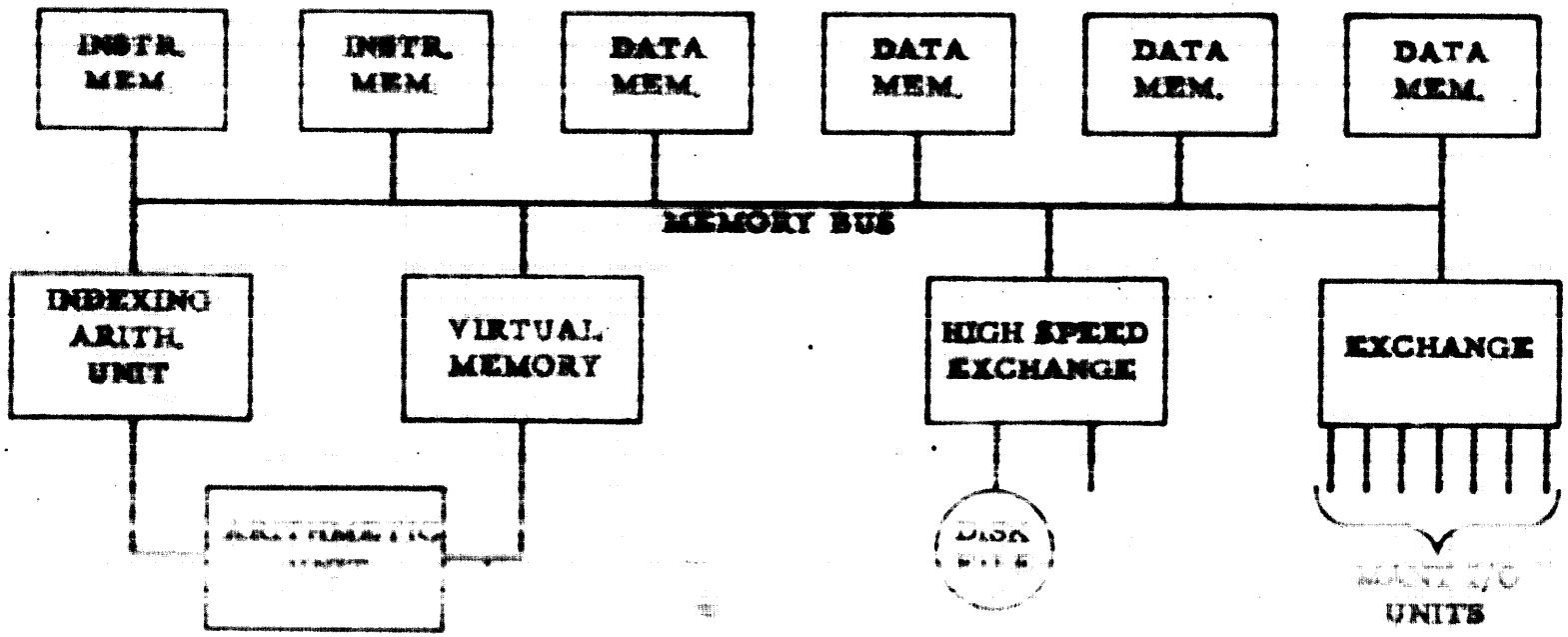
II. GENERAL DESCRIPTION OF THE SYSTEM

The major logically-independent blocks of the STRETCH computer are shown in the first slide (slide I). Each of the units pictured may be considered as operating asynchronously. That is, each does its tasks as fast as possible independently of the others. In theory, each box could have its own clocking circuits and still operate properly. In practice, for economy's sake they are all timed by the same master oscillator, but this does not destroy their logical independence.

The bus control unit serves as a routing agent between the memories and the various data processing units. If two or more units make a request

FIGURE 1

SCHEMATIC OF STRETCH COMPUTER



simultaneously the control unit assigns priorities giving the mechanical devices ~~in the following order:~~ ~~the~~ The highest priority (1) High speed Exchange, (2) Basic Exchange, (3) Virtual Memory, and (4) Indexing Arithmetic Unit.

The Indexing Arithmetic Unit fetches instructions, performs all necessary indexing operations and sends the instructions to be executed to the Virtual Memory.

The Virtual Memory fetches and receives the data required by the instruction and holds this data until the arithmetic unit is ready for it. The Virtual Memory also performs all store operations. It holds the data generated by the Arithmetic Unit ~~for Indexing Arithmetic Unit~~ until the memory to which the data must be sent is available. Thus the Virtual Memory acts not only as a "look-ahead" for instructions to be fed to the arithmetic unit, but also acts as a "look-behind" storage buffer.

next page

13 11

With the above general organization in mind, let us discuss some of the logical problems posed by such a system. The first problem is a result of the very concept which enables us to obtain such great benefits from the stored program computer -- the ability to treat instructions as data. In a system such as we have proposed there is a large amount of simultaneous operation. For example, the indexing arithmetic unit may be busy preparing an instruction before previous instructions have been completed or even started by the arithmetic unit. One of these previous instructions may modify the instruction which is presently being indexed. The virtual memory must recognize this situation and allow the intervening instructions to be completed before doing the modified instruction.

Consider the first Exemption Slide 2:

1 Since the instruction counter is normally about four instructions ahead of the arithmetic operations, " MPY C" would already be fetched from A+1 before the "ADD D" has been stored. The virtual Menmory must prevent the "MPY C" from being executed instead of the "ADD D". The instruction counter is compared with the Virtual Memory and no fetch is performed. The data can be transmitted from one Virtual Memory level to another.

A similar problem exists with respect to ordinary data. In order to operate several memories simultaneously, it is necessary to start obtaining data from these memories before the preceding operations have been completed. Yet, one of these operations may be a store into one of the data locations

For instance in Example 2.

1 If index registers 1 and 2 happen to contain the same value , then we we a problem similar to the previous instruction fetch case associated with the data fetch. Again a comparison causes the data to be forwarded to the appropriate Virtual Memory level. Note that it would be difficult for a program to determine in advance all the possible ways in which this type of trouble can occur, particularly in view of the fact that memory conflicts, etc. can vary the depth of look-ahead from one run to the next. This is particularly true when the instructions which must be compared need not be adjacent. They may be separated by one or two other instructions.

I. INSTRUCTION FETCH

A	LOAD B	WHERE C(3) ADD D
A + 1	STORE A + 2	
A + 2	MPY C	

II. DATA STORE

A	LOAD C
A + 1	STORE T, 1
A + 2	ADD T, 2

III. INTERRUPTION

A	ADD C, 1	
A + 1	INCR INDEX	→ B (OVERFLOW ROUTINE)
A + 2		B + 1

SLIDE 2

One of the novel features of the STRETCH computer is its elaborate interrupt system. Under this system whenever some unexpected occurrence arises, the program will be interrupted and control will pass to a special routine which is designed to take care of the case in question, ~~then return control~~ to the original program. In this situation the virtual memory must have provisions to retain enough information so that when an interrupt occurs we can resume the computation exactly where we left off. It must be able to recognize which of the changes that have been made in advance are not desired and should be obliterated and which are exact solutions that must be restored.

Example III

Example II

For example In this case let us assume that an overflow occurs on the ADD instruction. Under STRETCH's interrupt proceedure the control of the machine is trapped to a specified location. In this case it contains an OVERFLOW SUBROUTINE. The index operation in A+1 should not have been performed since the interrupt occurs. The index value must be replaced to its old value automatically.

Another special case arises when a conditional branch on arithmetic results occurs. Here we will not know which of the two branches we should have taken until the preceding instruction is executed. ~~In the case the wrong path has been selected,~~ the virtual memory must be prepared to drop the intermediate results which have been computed and pick up the correct branch in a way very similar to that of an interrupt. Note, however that index branches or unconditional branches do not have this problem since the IBox knows the right branch.

Summing up all these logical problems, we may state that the fundamental rule for the Virtual Memory is that it must make the asynchronous and non-sequential computer give results identical to those which would be obtained by performing the program one instruction at a time in the order in which they are written. In other words the programmer should not be aware that the virtual memory device exists in the machine!

(6 min)

(1) Technically speaking, the general Virtual Memory is a group of associative memory cells with control logic. When any data or instruction is to be fetched, its address is compared with the Virtual Menory to determine whether or not this data has already been placed in the Memory by a previous operation. If this has happened, the rule holds that data must be removed from the Virtual Memory instead of from the normal memory cell.

The structure of one of the Virtual Memory cells is shown on the next slide. Slide 3

(2) Level of Virtual Memory - A collection of registers and control bits. The contents of the j th level is shown in the slide.

(3) Instruction Address Register (I_j) - Contains the address of the instruction currently in the j th level.

(4) Operation Code Register (OP_j) - Contains the operation to be performed by the arithmetic unit.

(5) Store Bit (S_j) - a one bit trigger which indicates the level, contains a Store type instruction.

After Store Type

(2) Store Type - Instructions requiring the transmission of data from the Virtual Memory to external memory or index memory.

(Note: We consider all indexing instructions to be of the Store Type, although the store may be to either external memory or index memory.)

(6) Bring Bit (B_j) - A one bit trigger which indicates the level, contains a fetch type instruction for which the data access has not been started.

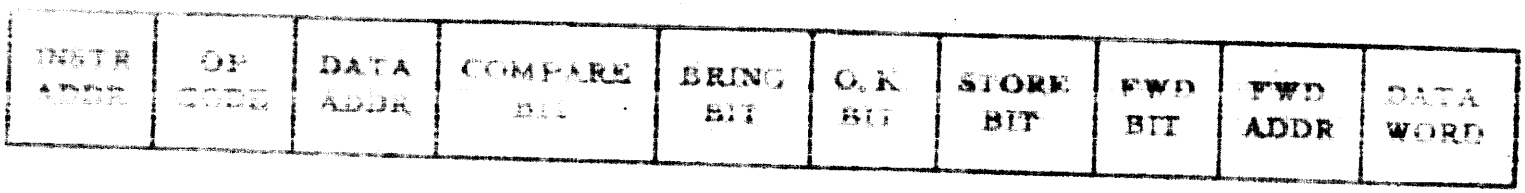
FETCH Type

(1) Bring or fetch type - All instructions requiring data to be transmitted from external memory to the Virtual Memory.

19 25

FIGURE 2

VIRTUAL MEMORY - CONTENTS OF ONE LEVEL



VIRTUAL MEMORY LOCATION COUNTERS

[illegible]

(7) Forwarding Bit (F_j) - A one bit trigger which indicates that the j th level must transmit data to another level.

(8) Forwarding Address (FA_j) - A register which contains the number of the level to which the data must be sent if F_j is set.

(9) O. K. Bit (OK_j) - A trigger which when set indicates that the correct data for the instruction to be executed is present in the j th Data Field.

(10) Data Field (D_j) - A register which contains the operand data for the instruction.

(11) Data Address (DA_j) - The operand data address (already indexed by the IAU) for D_j.

(12) Compare Bit (C_j) - A trigger which if not set indicates the address in DA_j should not be included in any address comparisons being made.

C. Logic of the Virtual Memory

1. General

There are two basic precepts which must be kept in mind to understand the operation of the Virtual Memory:

(1) The OK bit (OK_j) being set in the j th level indicates that the contents of the Data Field (D_j) is the correct data called for by Data Address (DA_j). All operations will be performed only under this condition and logical decisions will be made in such a manner as to make sure this is the case.

(2) Addresses can be compared by the IAU with every Data (DA_j) address simultaneously. The Data Address (DA_j) is not used for any level which does not have its C_j bit set. If a comparison exists between a new Data Address j (DA_j) being placed in the Virtual Memory and an old Data Address K (DA_k), the compare bit C_k is turned off and the address of level j is placed in Forward j Addr K (FA_k). This insures a unique meaning for the comparison. If this were not done, another instruction address DA_e might compare against two levels and thus cause an ambiguity.

3. Counters

The Virtual Memory is controlled by a set of counters which count mod(N), where N is the number of Virtual Memory levels.

(1) Counter one (C_1) - Indicates the level into which the next instruction may be placed.

(2) Counter two (C_2) - Indicates the level from which the next bring type instruction may be initiated.

(3) Counter three (C_3) - Indicates the level from which the next store type instruction may be initiated.

(4) Counter four (C_4) - Indicates the level from which the arithmetic unit will get its next operation and data.

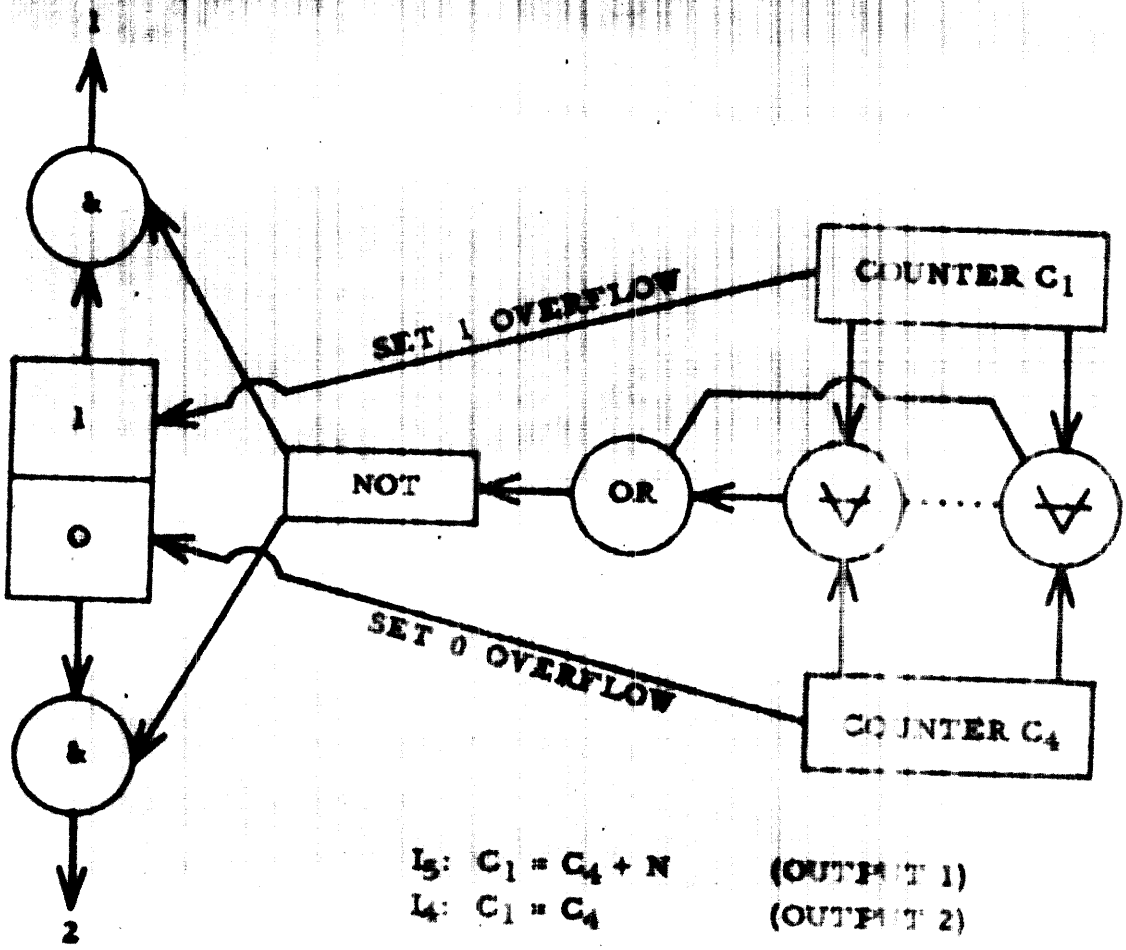
4. Interlocks

The above counters must be interlocked ~~in the following manner~~ to assure proper sequential operation of the computer (see figure 3):

Without going into detail, I can say that these interlocks prevent the contents of each level from being used before it is ready, or being destroyed before it is used. The example on the slide protects STORE-type instructions.

FIGURE 3

VIRTUAL MEMORY INTERLOCKS



I_5: $C_1 = C_4 + N$ (OUTPUT 1)

I_4: $C_1 = C_4$ (OUTPUT 2)

INTERLOCKS I_4 AND I_5 ARE AS SHOWN. THE OTHER INTERLOCKS ARE DONE IN A SIMILAR MANNER.

Slide 4

(11 min)

Hardware

The actual design of the Virtual Memory device posed a number of logical problems, particularly in connection with the examples mentioned earlier. I should emphasize here that the Concept just described is not meant to be an exact description of the STRETCH hardware. The principles are the same, or course, but practical considerations forced a number of changes in detail in the circuit design. with the inevitable loss of generality and symmetry.

Further, a machine organization of this complexity requires a detailed performance analysis in order to determine the value of adding hardware in the form of the "virtual memory". This is especially true since the sole function of the "virtual memory" is to increase machine speed, by increasing the efficiency of other devices. It was also felt that the timing analysis could not be made on the basis of a few trivial hand examples, (e.g. matrix multiply). Machine performance obtained in this fashion can be extremely deceptive. Since a detailed timing analysis of a computer of this complexity is extremely tedious to carry out by hand, It became clear that if the analysis were to be done, it would be necessary to simulate the proposed machine on another computer. This prompted us to write a timing simulation program for STRETCH on the IBM 704, to be described later.

(Slide 5)

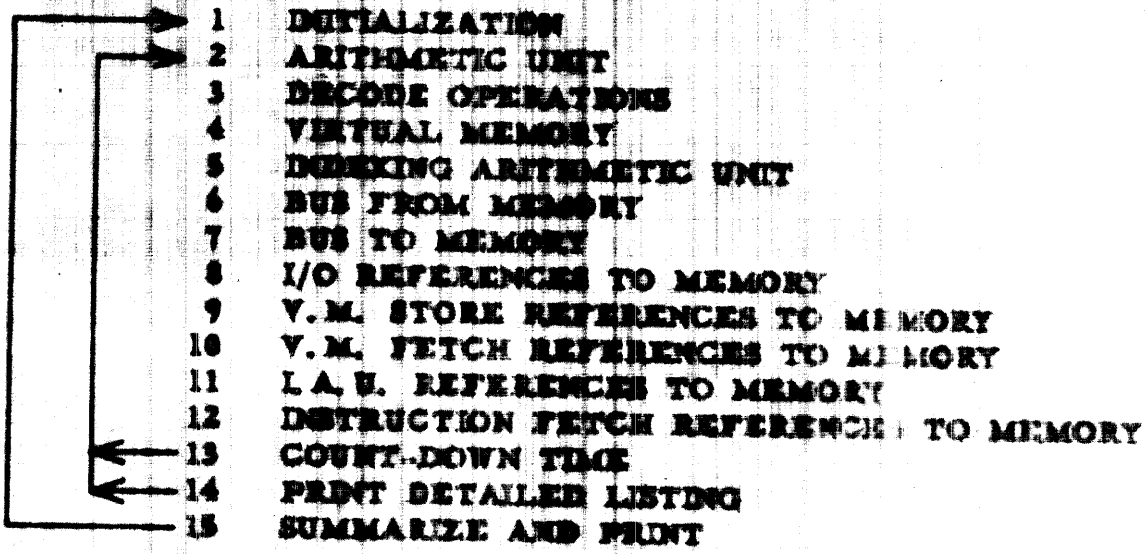
The basic logic of the 704 program follows the principles just described in the preceding section for the Virtual Memory. It should be stressed that the Simulator is a Timing Simulator and does not execute the instructions in an arithmetic sense. It traces the timewise progress of the instructions through the components of the computer observing all the interlocks and time delays necessary for correct representation of the behavior of the machine.

25

no p. 23-24

FIGURE 11

SIM - 2 SIMPLIFIED FLOW DIAGRAM



~~It is interesting to note that since the Simulator simulates tim-~~ To test the performance of a STRETCH configuration, an actual problem
~~ing only, not the arithmetic or indexing functions, the sequence of~~ ~~instructions~~ is coded and prepared ~~to be executed must be furnished~~ as a "string" with all loops unwound. ~~However,~~ To make the computer behave as it actually would, the loops must also be furnished with "wrong way" paths given for the cases where the computer would take such paths. ~~Also one must~~ furnish more than enough information along ~~such paths since it is difficult~~ to predict in advance how far the computer will get down the wrong path before it is called back.

Parameters are changed from one run to another by use of control cards. The control cards are set up in such a way that any number of parameters may be changed between runs. Results are given either as detailed timing charts or as summary listings for each problem. The usual procedure has been to print only summary results while making a series of parameter studies. The detailed timing charts as printed on the 704 for most problems would be about 50 feet long for each run. Since over a 1000 cases have been run, it is clear that only a few cases could be printed in full detail. These are particularly useful in seeking the causes of conflicts which slow the computer.

Table 5 (Timing chart)

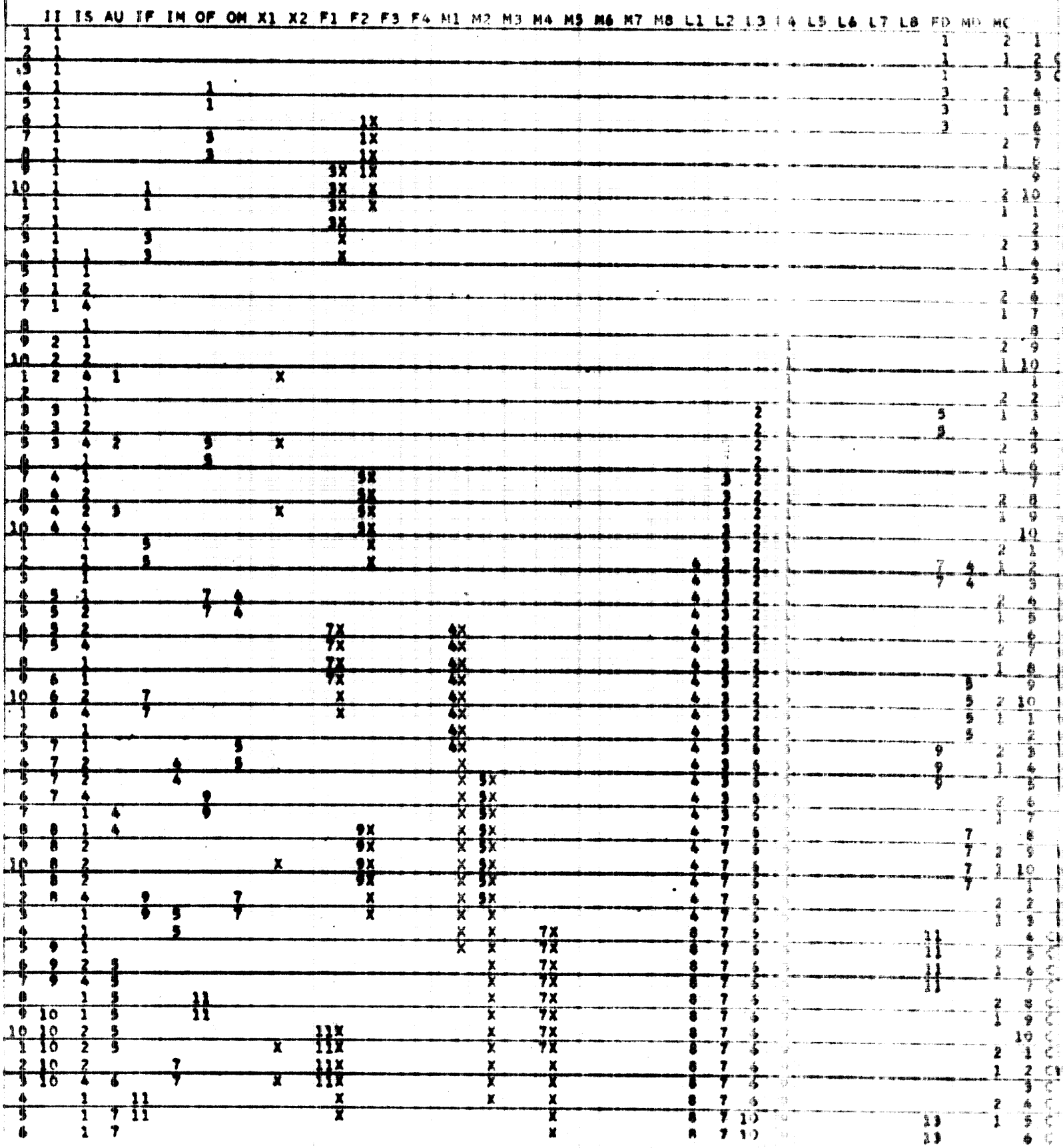
~~C. Results of Parameter Studies~~

When the Simulator Program was completed, we undertook a series of studies in which the main parameters describing the STRETCH system were varied one or two at a time in order to get

Figure 12
Listing of Simulator Print-Out

Slide 6

PROJECT 7000 SIMULATOR #2 COCKE + KOLSKY NOV 57



a measure for the importance of different effects. After this we began to specialize the studies towards answering specific questions in the STRETCH design. What we are presenting here are the results of some of the general parametric studies.

The simplified Flow Diagram in Figure 11, indicates the order in which the subroutines for the various logical units are executed at each time step. Using the types of techniques just described above, the logical subroutines simulate the action of the components of the computer such as the Virtual Memory, arithmetic Unit, etc.

4

Arithmetic and logic are about equally important. It is limited both by arithmetic and instruction-access speeds.

Speed vs Number of Levels of Virtual Memory

Figure 13 shows the effect on computer performance of varying the number of levels of Virtual Memory. Curves for the Monte Carlo and Mesh Calculations with two sets of arithmetic and indexing arithmetic speeds are shown. The AU times given are averages for all operations.

A number of interesting results are apparent from these curves:

(1) There is a tremendous gain to be had in going to the Virtual Memory organization. The point for "0 levels" means that the arithmetic unit is tied directly to the instruction preparation unit, although simple Indexing-Execution overlap is still possible.

(2) The gain in performance goes up very rapidly for the first two levels then rises more slowly for the rest of the range.

(3) A large number of levels does the Monte Carlo problem less good than the Mesh problem because constant branching in the former spoils the flow of instructions. Notice that the curve for the Monte Carlo problem actually decreases slightly beyond six levels. This phenomenom is a result of memory conflicts caused by extraneous memory references started by the computer running ahead on the wrong-way paths of branches.

Figure 13

COMPUTER SPEED

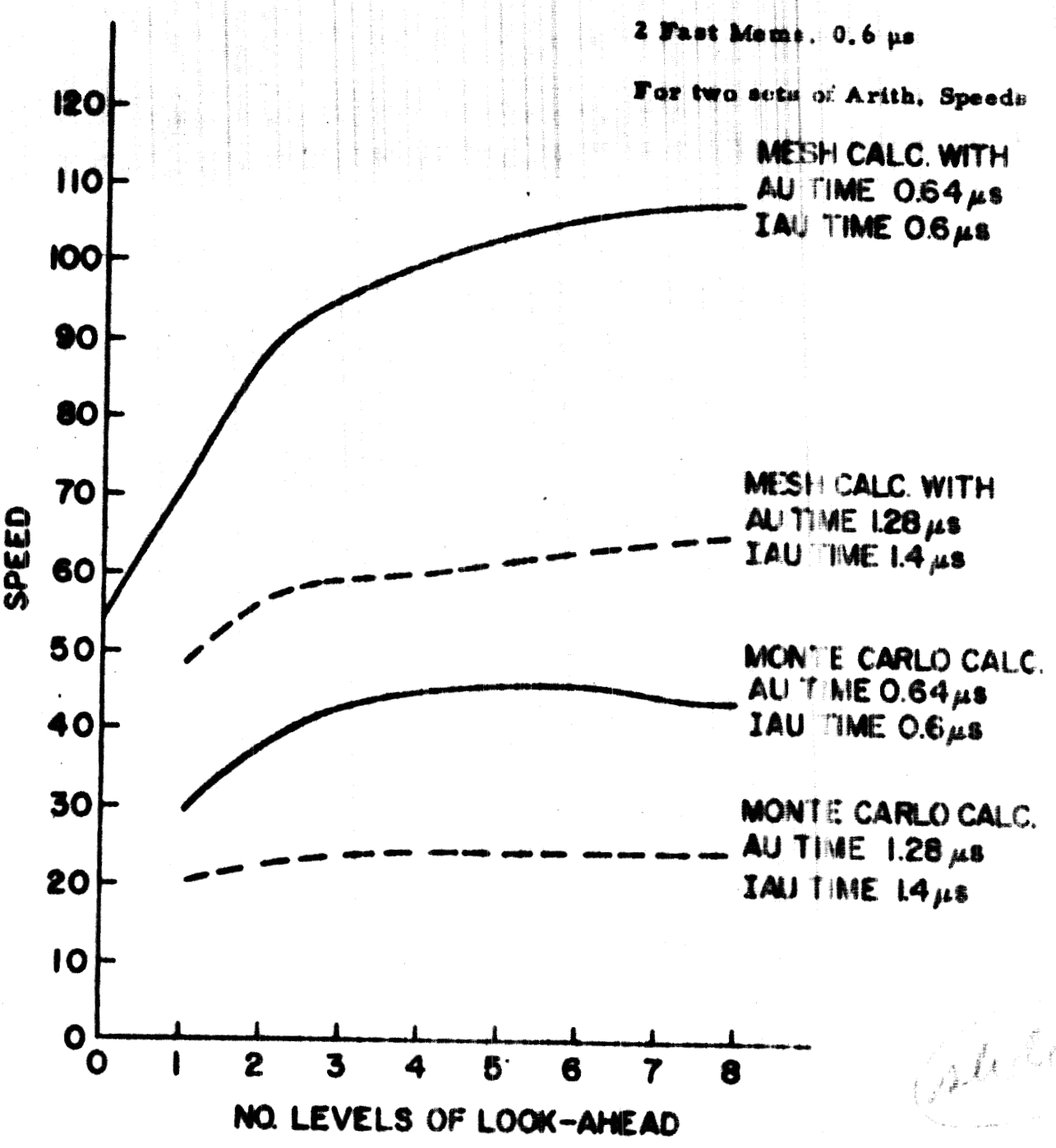
vs. No. of Levels of

Look-Ahead Registers

4 Main Mems. 2.0 μs

2 Fast Mems. 0.6 μs

For two sets of Arith. Speeds



(4) The computer performance on a given problem is clearly less for slower arithmetic speeds. However, it is important to note that the sensitivity of the performance is also less for slower arithmetic speeds. The Virtual Memory improves the performance in either case, but it is not a substitute for a fast arithmetic unit.

Speed vs Number of Main Memory Units

slide 8

slide 8

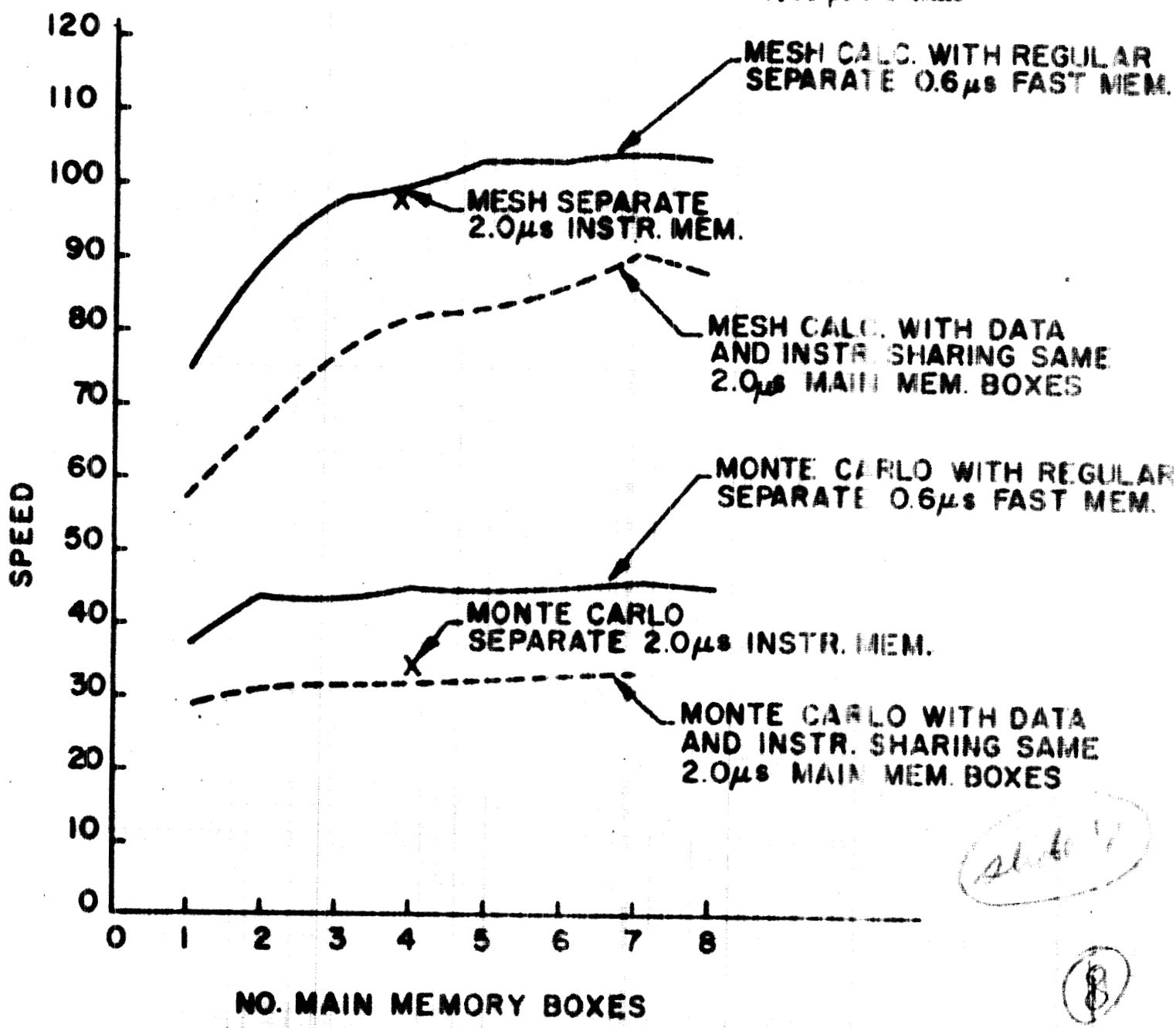
Figure 14 shows how internal computer performance varies with the total number of memory units for a particular problem. The entire calculation is assumed to be contained in memory for all cases. The speed gain from overlapping memories is quite apparent from the graphs.

The speed differential between having and not having instructions separated from data arises from delays in instruction fetches caused by the memory units being busy with data. The size of this effect varies from problem to problem, being less pronounced for problems which are arithmetic limited and more for logical problems.

The "X's" on the graph show the effect of replacing the 0.6 usec instruction memories by a pair of 2.0 usec memories. The resulting performance change is small for the Mesh Problem, which is arithmetic limited, but large for the instruction-fetch limited Monte Carlo problem.

Figure 4

COMPUTER SPEED
vs. Number of Main
Memory Boxes
4 levels of
0.6 μs IAU time
0.64 μs FXU time

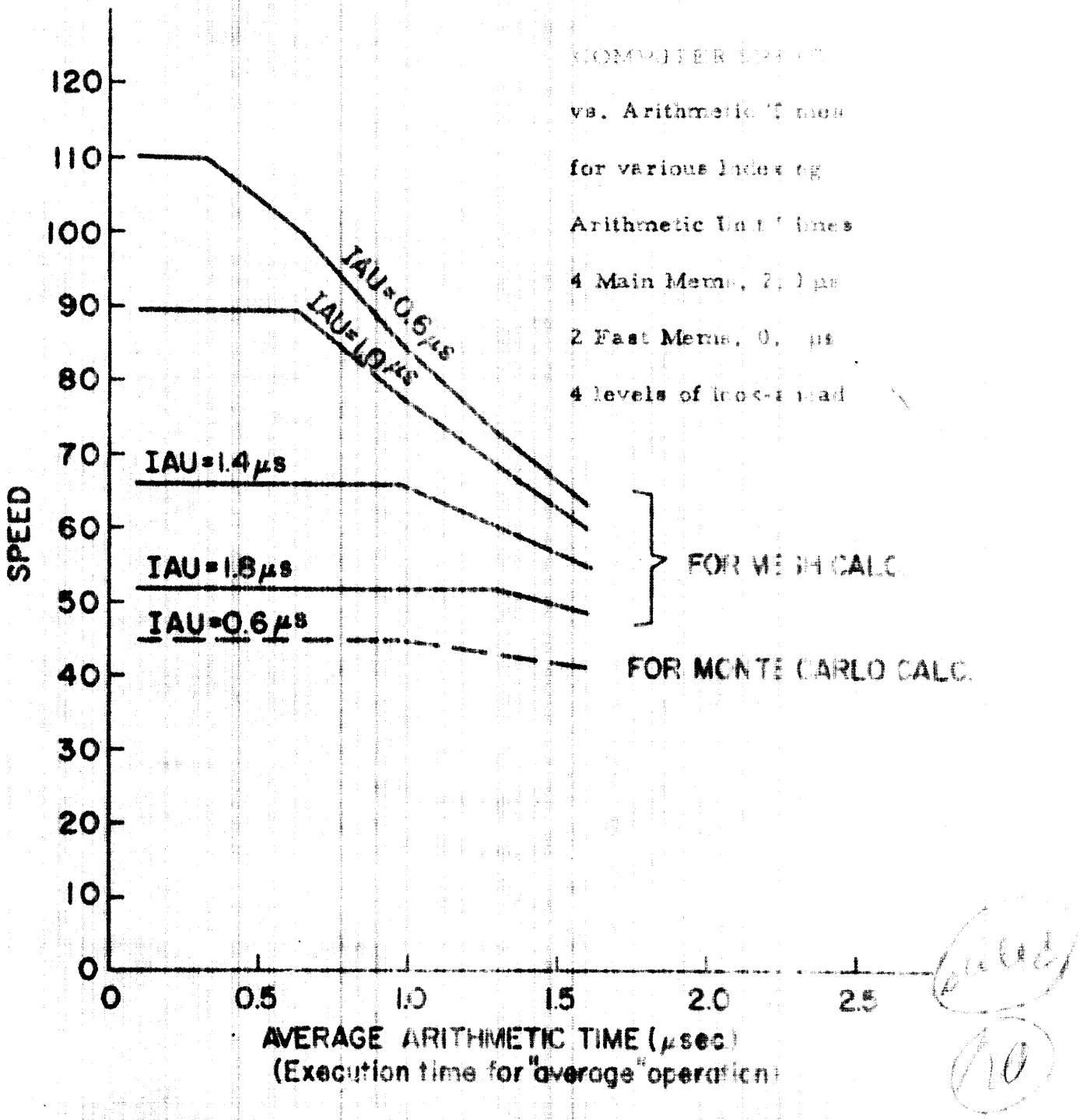


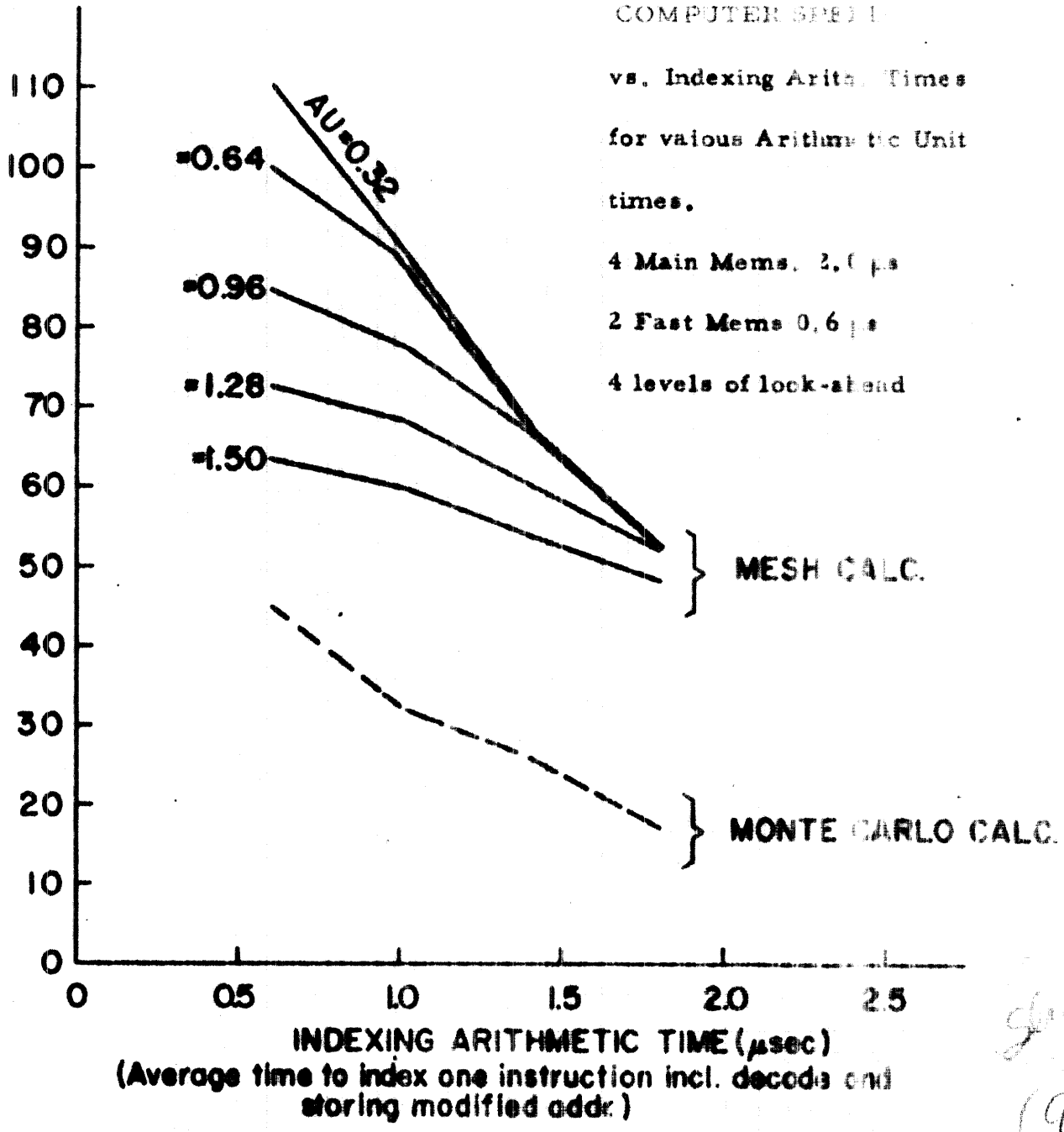
Speed vs Arithmetic Unit and Indexing Arithmetic Unit Times

Although everyone realizes the importance of arithmetic speed on overall computer performance, it was not until the Simulator results became available that the true importance of the indexing arithmetic speeds was recognized. Figures 15 and 16 show a two parameter family of curves giving the computer speed as a function of the AU and IAU times. Figure 16 in which the arithmetic time is the abscissa shows an interesting "saturation" effect where the computer performance is independent of AU speed below some critical value. Thus it makes no sense to strain AU speeds if the IAU is not improved to match. The curves in Figure 15 show the same effect i. e., the IAU speed serves as a "ceiling" on performance beyond which the AU speed cannot pass.

Arithmetic Unit Efficiency

One fallacy which is frequently quoted is that the goal of improved computer organization is to increase the arithmetic unit efficiency. Actually there are two reasons why this is not the goal in itself. The first is that arithmetic efficiency depends strongly on the mixture of arithmetic and logic in a given problem so that a general purpose computer cannot hope to give equally high percentage utility to all. The second reason is that the simplest way to increase the arithmetic unit efficiency in any asynchronous case is to slow down the arithmetic unit!


COMPUTER SPEED
vs. Arithmetic Time
for various Index
Arithmetic Unit Times
4 Main Memo, 2.1 μs
2 Fast Memo, 0. μs
4 levels of Index Read
IAU=0.6μs
IAU=1.0μs
IAU=1.4μs
IAU=1.8μs
IAU=0.6μs
FOR MESH CALC
FOR MONTE CARLO CALC.
SPEED
120
110
100
90
80
70
60
50
40
30
20
10
0
0
0.5
1.0
1.5
2.0
2.5
AVERAGE ARITHMETIC TIME (μsec)
(Execution time for "average" operation)


COMPUTER SPEED
vs. Indexing Arith. Times
for vaious Arithmetic Unit
times.
4 Main Mems. 2.0 μs
2 Fast Mems 0.6 μs
4 levels of look-ahead
AU=0.32
=0.64
=0.96
=1.28
=1.50
MESH CALC.
MONTE CARLO CALC.
110
100
90
80
70
60
50
40
30
20
10
0
0
0.5
1.0
1.5
2.0
2.5
INDEXING ARITHMETIC TIME (μsec)
(Average time to index one instruction incl. decode and storing modified addr.)

(9)

The real goal of improved organization is maximum overall computer performance for minimum cost. One will tend to increase the arithmetic unit speed as long as its percent efficiency is reasonable for a variety of problems. One will stop this process when the overall performance gain no longer matches the increase in hardware and complexity. Thus the arithmetic unit efficiency is a by-product of this design process not the prime variable.

Speed vs Concurrent Input-Output Activity

Because of the relative time scales of I/O activity and the CPU processing speeds the Simulator cannot take in account the availability or non-availability of data from I/O on the program being run. However, Using The Simulator it was interesting to we can observe the effect on the computation of the I/O devices operating at different rates simultaneously with computing.

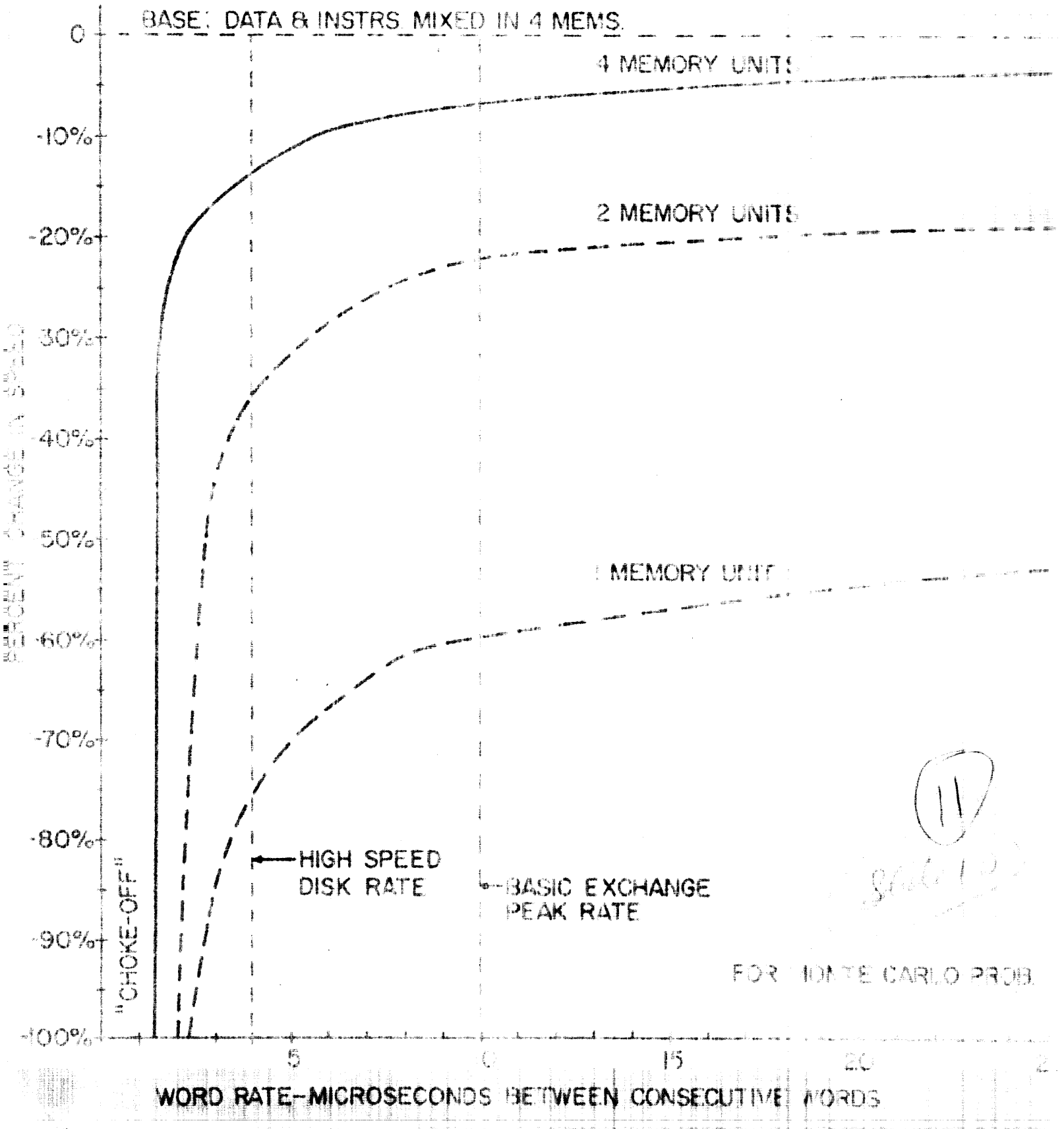
With Using the STRETCH control word philospphy it is possible to have a number of input-output units operating at the same time the Central Processing Unit is running. The Basic Exchange can reach a peak rate of 1 word every 10 microseconds. The high speed disk normally operates at 1 word every 4 microseconds. Since the mechanical devices take priority over the CPU in addressing memory, the computation slows down because of memory-busy conflicts.

The Table Figure 17 shows an example of how internal computing speed is slowed as the I/O word rates are varied continuously. At the theoretical "choke off" the I/O devices take all the memory cycles available and stop the calculation. Notice that this condition can never arise for any I/O rates presently attainable.

51

Figure 17

Percentage Reduction in Speed caused by Input-Output devices referencing memory at different rates while the calculation is proceeding.



Notice that

A STRETCH system with only 1 or 2 Memory Units has ~~less~~ lower performance than a larger one for three reasons: (1) The top speed of the system is reduced by the loss of memory overlap, (2) It has a larger I/O penalty when I/O is run concurrently with the computation, and (3) The smaller amount of data which can be held in the memory at one time increases the amount of I/O activity needed to do the job. Note, however, that increasing the memory size on a computer of conventional organization would only improve the third area.

The Virtual Memory is thus very effective in smoothing out the Input-Output demands on the memory units so that the arithmetic units can still maintain quite high performance under a veritable avalanche of I/O activity.

Those of you who would like to know more about the wonders of the Virtual Memory will find a much more detailed write-up in the Proceedings. Thank you..

End

(actual meas)

120. Introduction to STRICT

~~200~~ 170

~~130~~ 110 400 ~~450~~ 3m ~~4~~ (3)

~~300~~ 230 Logical Probs

~~290~~ 280 510 ~~540~~ 3 ~~4~~m 8 (6)

~~240~~ 250 Detailed descript. of VM

160

260

~~220~~ 110 780 ~~930~~ 5 ~~6~~m. ~~17~~ (11)

~~250~~ 230 descript. of SIM

~~140~~ 0

~~220~~ 180 X

40 450 ~~650~~ 3 ~~4~~m 17 (14)

190 Results of SIM Runs

150

~~140~~ 130

210 810 5

130 ~~870~~ ~~6~~m (19)

~~3490~~ ~~2950~~ (23) 24

60